

Robot Framework E-Commerce Automation

Capstone Assignment 4 -- Keyword-Driven and Data-Driven Testing

Capstone Assignment 4

Robot Framework Web Automation Project

Target application: automationexercise.com

Report generated as part of the capstone submission

Table of Contents

- 1. Business Scenario and Objective
- 2. Requirement Coverage Map
- 3. Architecture and Design Decisions
- 4. Unique Value-Add Feature: Cross-Run History Dashboard
- 5. Real Issues Found and Fixed During Development
- 6. Test Execution Demonstration
- 7. How to Run (Command Line and Jenkins)
- 8. Conclusion

1. Business Scenario and Objective

The assignment calls for a complete Robot Framework automation solution covering the business flow: Launch Browser -> Login -> Search Product -> Add Product To Cart -> Verify Cart -> Logout -> Close Browser, against automationexercise.com. This is a separate, standalone project from the Selenium/PyTest framework built for Assignment 2 -- same target site, deliberately different tooling and design, per the requirement that the two capstone projects be independent of each other.

2. Requirement Coverage Map

Brief Requirement	Where It Lives	Notes
SeleniumLibrary	resources/*.resource	All browser interaction goes through SeleniumLibrary keywords.
Keyword-Driven Testing	resources/pages/*.resource	
Data-Driven Testing	tests/end_to_end_shopping.robot (Test Template), tests/login.robot ([Template])	Tests call named keywords (Login With Credentials, Search For Product) -- never a raw locator.
Resource Files	resources/common.resource, resources/pages/*.resource	See Section 5 for the real difference between these two mechanisms, discovered mid-development.
User Defined Keywords	Every *.resource file	Locators and keywords are never duplicated inside a test suite.
Setup and Teardown	Suite Setup/Teardown (account), Test Setup/Teardown (browser) in both suites	e.g. Click Element Safely, Cart Should Contain Product.
Page Object Model Concept	resources/pages/	
Jenkins Execution	Jenkinsfile	One resource file per page: locators as Variables, actions as Keywords.
Command Line Execution	README.md	Installs deps into a venv, runs the suite, archives results/ regardless of outcome.
Reporting	results/log.html, report.html, output.xml (Robot Framework); results/dashboard.html (custom, see Section 4)	robot --outputdir results --listener libraries/HistoryListener.py tests/

3. Architecture and Design Decisions

Page Object Model, expressed as Robot resource files

Each page on the site owns its own resource file (home_page.resource, login_page.resource, product_page.resource, cart_page.resource): locators as `*** Variables ***`, actions as `*** Keywords ***`. Test suites never reference a CSS selector directly -- the same separation of concerns POM gives in code, in Robot's own resource-file mechanism.

A native Robot library for the test-account constraint

automationexercise.com has no fixed demo login -- the same real constraint the Selenium/PyTest project faces. `libraries/AccountApi.py` solves it the same way (calling the site's own `createAccount/deleteAccount` REST endpoints) but as a native Robot Framework Python library with `ROBOT_LIBRARY_SCOPE = SUITE`, so Suite Setup and every test case in that suite share one provisioned account.

4. Unique Value-Add Feature: Cross-Run History Dashboard

Robot Framework's own log.html and report.html only ever describe a single run. What matters once a suite is wired into CI is which tests are reliably passing and which ones fail intermittently -- and that only becomes visible by looking across many runs. `libraries/HistoryListener.py` implements Robot's Listener API v3 and is wired in via the command line (`--listener`), not imported by any resource or test file, since a listener is run configuration rather than test logic.

- `end_test(data, result)` appends one JSON line per test per run to `results/history.jsonl`.
- `close()` runs once the entire execution finishes, aggregates every run ever recorded, and writes `results/dashboard.html` ranking tests by failure rate.

This was verified by running the suite twice in a row without clearing the history file: the dashboard correctly reported 2 runs recorded for every test, confirming the listener persists state correctly across independent Robot executions -- exactly the behaviour a CI pipeline running this suite nightly would rely on.

5. Real Issues Found and Fixed During Development

Issue 1 -- [Template] on one test case silently shares Setup/Teardown across all its data rows

The first version of the end-to-end suite put [Template] directly on a single test case with three search-term rows (Top, Dress, Tshirt). Running it produced only ONE test in the summary instead of three, and it failed with the cart containing BOTH 'Blue Top' and 'Sleeveless Dress' -- items from two different search terms. A minimal isolated experiment confirmed the cause: a [Template] on one test case folds every data row into that same test, so Test Setup/Teardown ran once for all three rows combined, meaning all three search terms shared one browser and one login session. Fix: switched to a suite-level Test Template with three separately named *** Test Cases *** rows, giving each search term its own Test Setup/Teardown.

Issue 2 -- Ad iframes intercept clicks anywhere on the page, not just one button

Even after fixing Issue 1, individual runs still failed intermittently with `ElementClickInterceptedException` -- first on the add-to-cart button, then on the 'View Cart' modal link, then on the search button and the login button in later runs. The site's Google ad slots can overlap any clickable element. Fix: a shared 'Click Element Safely' keyword (resources/common.resource) tries a normal click and falls back to a JavaScript click on `ElementClickInterceptedException`; every click in the project was switched to use it.

Issue 3 -- Inconsistent whitespace in search-result product names

A cart-content assertion failed with a text mismatch: the expected substring 'Sleeveless Dress' (double space) did not match the cart's actual text 'Sleeveless Dress' (single space). Live inspection confirmed the search-results grid genuinely renders that product name with a double space, while the plain /products grid does not. Fix: Get Product Result Names now normalizes whitespace with `Evaluate ' '.join(text.split())` before returning names.

Issue 4 -- A wrong keyword name and a stale-element race

An early draft called a non-existent keyword, 'Element Text Should Contain' -- SeleniumLibrary only provides 'Element Should Contain'. Separately, asserting the post-login 'Logged in as' label with a Wait-then-assert pattern occasionally raised `StaleElementReferenceException`, because the nav bar was still settling after the redirect. Fix: the correct keyword name, and 'Wait Until Element Contains', which re-locates the element on every retry instead of asserting against a handle grabbed a moment too early.

6. Test Execution Demonstration

Actual terminal output from a full run of both suites together against the live site:

```
=====
Tests.End To End Shopping :: End-to-end shopping flow: login, search for a ...
=====
Shop For A Top | PASS |
-----
Shop For A Dress | PASS |
-----
Shop For A Tshirt | PASS |
-----
Tests.End To End Shopping :: End-to-end shopping flow: login, sear... | PASS |
3 tests, 3 passed, 0 failed
=====
Tests.Login :: Login functionality: a valid account plus a set of data-driv...
=====
Valid Login Succeeds | PASS |
-----
Login With Wrong Password Is Rejected | PASS |
-----
Login Rejected For Invalid Credentials | PASS |
-----
Tests.Login :: Login functionality: a valid account plus a set of ... | PASS |
3 tests, 3 passed, 0 failed
=====
Tests | PASS |
6 tests, 6 passed, 0 failed
```

Running the same command a second time without clearing results/history.jsonl confirmed the History Dashboard feature works across independent executions:

```
Test history dashboard written to .../results/dashboard.html (12 recorded test run(s))
```

(6 tests x 2 runs = 12 recorded entries, exactly as expected.)

7. How to Run (Command Line and Jenkins)

Command line

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

robot --outputdir results --listener libraries/HistoryListener.py tests/
```

Jenkins

The included Jenkinsfile defines a two-stage declarative pipeline: install dependencies into a fresh virtual environment, then run the same Robot command. A post/always block archives every file under results/ (HTML reports, XML output, and any failure screenshots) regardless of whether the build passed, so a red build still leaves full evidence behind.

8. Conclusion

This project satisfies every bullet in the Assignment 4 brief using Robot Framework's own idioms rather than porting patterns from the Selenium/PyTest project: resource files for POM, native user-defined keywords, Suite/Test Setup and Teardown, and a Robot Listener for the one added value-add feature, a cross-run flakiness dashboard. Four real, distinct bugs were found and fixed by running the suite against the live site rather than assumed correct on paper, each documented in Section 5 with the execution evidence that proves the fix in Section 6.